Practical Lab: Introduction to Web Development with Python

Django & Flask Fundamentals

Course: Object-Oriented Programming with Python

April 2026

1. Lab Overview

This laboratory session introduces students to modern Python web development using two major frameworks: **Flask** (lightweight) and **Django** (full-stack).

Students will learn how to:

- Create and run a basic web server
- Understand routing and request handling
- Build simple web applications
- Compare Flask and Django architectures

2. Learning Objectives

By the end of this lab, students should be able to:

- Set up a Python web development environment
- Create a basic Flask application
- Create a basic Django application
- Understand the MVC/MVT architecture
- Run and test web applications locally

3. Prerequisites

- Python 3.8+
- Basic Python knowledge
- Command line usage
- Internet connection for package installation

4. Environment Setup

4.1 Create Virtual Environment

```
python -m venv venv

# Activate environment
# Windows
venv\Scripts\activate

# Linux/Mac
source venv/bin/activate
```

Why this matters: It isolates project dependencies.

5. Part I: Flask Framework

5.1 Introduction to Flask

Flask is a lightweight Python framework used for building simple and scalable web applications.

5.2 Install Flask

```
pip install flask
```

5.3 Project Structure

```
flask_app/
 app.py
 templates/
 static/
```

5.4 Create Flask Application

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Flask - HND Lab"

if __name__ == "__main__":
    app.run(debug=True)
```

5.5 Run Flask App

```
python app.py
```

Open browser:

http://127.0.0.1:5000

5.6 Concept Check

- Flask creates a local server
- Routes map URLs to functions
- Each function returns a response

6. Part II: Django Framework

6.1 Introduction to Django

Django is a full-stack framework designed for large and secure applications.

6.2 Install Django

```
pip install django
```

6.3 Create Project

```
django-admin startproject myproject
cd myproject
```

6.4 Project Structure

```
myproject/
 manage.py
 myproject/
 myapp/
```

6.5 Run Server

```
python manage.py runserver
```

6.6 Create Application

```
python manage.py startapp myapp
```

6.7 Migrations (IMPORTANT)

```
python manage.py migrate
python manage.py makemigrations
```

6.8 Simple View

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Django - HND Lab")
```

6.9 URL Mapping

```
from django.urls import path
from myapp.views import home

urlpatterns = [
    path('', home),
]
```

7. Flask vs Django Summary

Feature	Flask	Django
Type	Micro-framework	Full-stack
Database	Optional	Built-in ORM
Complexity	Easy	Medium
Best for	Small apps	Large systems

8. Common Errors & Fixes

- Flask not running check port 5000
- Django page not found check URL routing
- Module not found activate virtual environment

9. Best Practices

- Always use virtual environments
- Separate logic and presentation
- Keep code modular
- Test server after every change

10. Lab Checklist

Task	Done
Virtual environment created	[ ]
Flask app runs	[ ]
Django app runs	[ ]
Routing understood	[ ]
Server tested	[ ]

Conclusion

This lab provides the foundation for Python web development using Flask and Django. Students are encouraged to extend these applications with templates, databases, and authentication systems.